

WEEK 3:

KOTLIN

1. List vs Array
2. Classes
 - a. Regular classes
 - b. Data classes
 - c. Sealed classes
 - d. Open classes
 - e. Abstract classes
 - f. Interface
 - g. Enum
 - h. Nested classes
 - i. Inner classes
 - j. Singleton classes (Object)
 - k. Companion object
 - l. Open class vs Abstract class vs Interface
3. Visibility modifiers
 - a. public
 - b. private
 - c. protected
 - d. internal
4. Functions
 - a. Top level functions
 - b. Member functions
 - c. Extension functions
 - d. Inline functions
 - e. Lambda functions
 - f. Higher order functions
 - g. Local functions
 - h. Anonymous functions
 - i. Suspend functions
 - j. Infix functions
 - k. Operator functions
 - l. Tail recursive functions
 - m. Scope functions
- i. let
 - ii. with
 - iii. run
 - iv. apply
 - v. also
 5. Kotlin coroutines
 - a. Coroutines
 - b. Suspend functions
 - c. Builders
 - i. launch
 - ii. async
 - iii. runBlocking
 - iv. coroutineScope
 - v. supervisorScope
 - d. Dispatchers
 - i. Dispatchers.Main
 - ii. Dispatchers.IO
 - iii. Dispatchers.Default
 - iv. Dispatchers.Unconfined
 - e. withContext()
 - f. Processes
 - g. Threads
 6. Delegation
 7. Observables
 - a. LiveData
 - b. Flow
 8. Generics
 - a. Variance
 9. Operators
 - a. Arithmetic operators
 - b. Relation operators
 - c. Assignment operators
 - d. Unary operators
 - e. Logical operators
 - f. Bitwise operators

1. Hello world a. Variables b. String templates 2. Basic types a. Integers b. Unsigned integers c. Floating-point numbers d. Booleans e. Characters f. Strings 3. Collections a. Lists b. Sets c. Maps 4. Control flow a. Conditional expressions i. If ii. When b. Ranges c. Loops i. For ii. While iii. Do-while 5. Functions a. Named arguments b. Default parameter values c. Functions without return d. Single-expression functions e. Early returns in functions f. Lambda expressions i. Pass to another function ii. Function types iii. Return from a function iv. Invoke separately v. Trailing lambdas 6. Classes a. Properties b. Create instance c. Access properties	d. Member functions e. Data classes i. Print as string ii. Compare instances iii. Copy instance 7. Null safety a. Nullable types b. Check for null values c. Use safe calls d. Use elvis operator
---	--

1. Introduction a. Functions I. Default parameter values and named arguments II. Infix functions III. Operator functions IV. Functions with vararg parameters b. Variables c. Null safety d. Classes e. Generics I. Generic classes II. Generic functions f. Inheritance I. Inheritance II. Inheritance with parameterized constructor III. Passing constructor arguments to superclass	II. Object expression III. Object declaration IV. Companion objects
2. Control flow a. When I. When statement II. When expression b. Loops I. For II. While III. Do while IV. Iterators c. Ranges d. Equality checks e. Conditional expressions	4. Functional a. Higher order functions b. Lambda functions c. Extension functions and properties
3. Special classes a. Data classes b. Enum classes c. Sealed classes d. Object keyword I. Object keyword	5. Collections a. List b. Set c. Map d. filter e. map f. any, all, none g. find, findLast h. first, last i. count j. associateBy, groupBy k. partition l. flatMap m. minOrNull, maxOrNull n. sorted o. Map Element Access p. zip q. getOrElse
	6. Scope functions a. Let b. Run c. With d. Apply e. Also
	7. Delegation a. Delegation pattern b. Delegation properties I. Delegation properties II. Standard delegates III. Storing properties in a map
	8. Productivity boosters a. Named arguments

- | | |
|--|--|
| <ul style="list-style-type: none">g. String templatesh. Destructuring declarationsi. Smart casts | |
|--|--|

1. Creating a Variable	40. Inner Classes
2. Creating a Read-only Variable	41. Creating an Enum
3. Providing a Type on a Variable	42. Enum Constructor Parameters
4. Basic Types: Numbers	43. Abstract Functions in Kotlin
5. Basic Types: String and Char	44. Iterating over Enum Values
6. String Interpolation and Triple Quotes	45. Adding Static Methods to Enums
7. Basic Types: Boolean	46. The When Statement (Kotlin Switch Statement)
8. If/Else Conditionals	47. Exhaustive vs Non-Exhaustive When
9. One Line If/Else Conditional	48. Data Classes
10. Understanding Truth Tables	49. Data Class with Parameterless Constructor
11. Structural Equality	50. Data Class: Component1-N functions
12. Referential Equality	51. Destructuring Data Classes
13. Nullable Types	52. Copying Data Class Instances
14. Nullable Safe Calls	53. The Pair and Triple Data Classes
15. The Elvis Operator	54. Protected Modifier for Variables
16. Your First Kotlin Function	55. Protected Modifier for Functions
17. Functions with Return Types	56. Internal Modifier for Classes
18. Functions inside of Functions	57. Abstract Classes
19. Single Line Expressions	58. Implementing an Abstract Class
20. Function Arguments	59. Understanding Abstractions with Abstract Classes
21. Named Parameters	60. What interfaces are used for
22. Default Function Arguments	61. Creating an Interface
23. Multiple Args with varargs	62. Implementing an Interface
24. Function Overloading	63. Real World Interface Example
25. Creating your First Kotlin Class	64. Anonymous Interfaces
26. Class Primary Constructors	65. Creating Arrays
27. Multiple Class Constructors	66. Creating Immutable Lists
28. Constructor Init Blocks	67. Creating Mutable Lists
29. Class Properties	68. Filtering a List
30. Read Class Properties	69. Finding Items in a List
31. Overriding a Property Getter	70. Using "filterNot" on a List
32. Overriding a Property Setter	71. filterTo and filterNotTo on Lists
33. Multiple Properties in a Class	72. Flattening Lists and Arrays
34. Class Functions	73. Combining Immutable Lists
35. Companion Objects	74. List vs Map
36. How to Create a Singleton	
37. Declaring Constants	
38. The lateinit modifier	
39. How to Nest Classes	

75. Map vs FlatMap	107. Creating a typealias
76. Set Data Structure (a List with no Duplicates)	108. Extension Functions
77. For Loops	109. Lazy Evaluation
78. While Loops	110. Lazy Initializer Block
79. For Each Loops	111. Type Inference
80. Combining Lists with the Union Operator	112. Simple Lambda Expressions
81. Iterating over a List with an Index	113. Lambdas as Function Parameters
82. Ranges	114. Passing Values to Lambda Function Parameters
83. Immutable Maps	115. Underscore Parameters in Lambda Functions
84. Mutable Maps	116. The 'it' Parameter in Lambda Functions
85. Filtering and Transforming Maps	117. Calling Kotlin from Java
86. Using mapNotNull on a Map	118. Calling Java from Kotlin
87. Generating Large Sequences	119. Calling an Extension Function from Java
88. Measuring Performance	120. @JvmName Annotation
89. Kotlin Sequences	121. Top Level Main Functions
90. List vs Set vs Map vs Sequence	
91. Kotlin Ternary Operator: Single Line If	
92. Double Bang !! Operator	
93. Using requireNotNull()	
94. Using checkNotNull()	
95. Filtering a List with filterNotNull()	
96. Type Checking with 'is'	
97. Casting	
98. Safe Casting with 'as'	
99. Generic Lists and Maps	
100. Your First Generic Class	
101. Throwing Exceptions	
102. Creating a Custom Exception	
103. Catching Exceptions with Try/Catch	
104. Catching Multiple Exception Types	
105. Try/Catch/Finally blocks	
106. Try/Catch vs Try/Finally	

LIST	ARRAY
• listOf() • mutableListOf() • emptyList()	• arrayOf() • arrayOfNulls() • emptyArray() • Array<Int>(3) { 0 } • IntArray(3) • Array(2) { Array<Int>(2) { 0 } } • Array(2) { IntArray(2) }
• List<String> • MutableList<String> • List<Any>	• Array<String> • Array<Array<Int>> • Array<out Any> • BooleanArray • ByteArray • CharArray • DoubleArray • FloatArray • IntArray • LongArray • ShortArray
• List - Read-only, meaning you can't modify its elements after creation • MutableList- Allows modification of its elements	• Arrays are mutable by default, meaning you can change the value of elements but can't change the size of the array after it's created
• Lists can grow or shrink dynamically, depending on the implementation	• The size of an array is fixed once it is created
• You can access elements by index using [index]	• You can access elements by index using [index]
• Allow for duplicate items	• Allow for duplicate items
• Designed to hold elements of the same data type, ensuring type safety. However, you can create a List that holds elements of different data types by using a more general type like Any, which is the supertype of all Kotlin types	• Designed to hold elements of the same data type. However, you can also create arrays that hold elements of different types by using the Array<Any> type
• list	• simpleArray.joinToString() • threeDArray.contentDeepToString()

Classes

1. Regular classes

- The most basic type of class.

2. Data classes

- Special classes used for holding data, automatically providing methods like *toString()*, *equals()*, *hashCode()*, and *copy()*.

3. Open classes

- By marking a class with the `open` keyword, you allow it to be subclassed.

4. Abstract classes

- Classes that cannot be instantiated and are meant to be subclassed.
- They can have abstract members that must be implemented in subclasses.

5. Interface

- Like abstract classes but cannot hold state.
- Interfaces can contain abstract methods as well as method implementations.

6. Sealed classes

- Classes that represent a restricted hierarchy (Only subclasses defined within the same file can inherit from the sealed class), allowing a limited set of subclasses.
- Useful for representing a fixed set of types.
- Sealed classes are ideal for representing a state machine or other situations where you have a finite set of possible states.

7. Enum

- Classes that represent a fixed set of constants.

8. Nested class

- Classes defined within another class but cannot access the members of the outer class unless they are marked as `inner`.

9. Inner class

- Classes that are nested within another class and can access the members of the outer class.

10. Singleton classes (Object)

- Kotlin's way of creating a singleton class, meaning only one instance of this class can exist.

11. Companion object

- An object that is associated with a class and can access its private members.

OPEN CLASS	ABSTRACT CLASS	INTERFACE
open class	abstract class	interface
Methods with implementations	Methods with and without (abstract fun) implementations	Methods with and without implementations
Should not have method without implementation	Should override method without implementation	Should override method without implementation
Can override methods with implementations	Cannot override methods without implementations	Can override methods with implementations
Can create an instance	Cannot create an instance	Cannot create an instance
Can have a constructor	Can have a constructor	Cannot have a constructor
Single inheritance	Single inheritance	Multiple inheritance
Can have state, meaning it can hold properties with a backing field	Can have state, meaning it can hold properties with a backing field	Cannot hold state, meaning it cannot have properties with a backing field

Visibility modifiers

1. **public**

- Accessible from anywhere.

2. **private**

- Accessible only within the same class or file.

3. **protected**

- Accessible within the same class and its subclasses.

4. **internal**

- Accessible within the same module.

Functions

1. Top level functions

- These functions are defined outside of any class, object, or interface.

2. Member functions

- Functions that are defined within a class or object.

3. Extension functions

- Functions that add new functionality to existing classes without modifying their source code.

4. Inline functions

- The compiler replaces the function call with the actual code of the function.
- Normally, when you call a function, the program needs to jump to the function's code, execute it, and then return to the place where it was called.
- With an inline function, instead of jumping to the function, the program directly puts the code of the function wherever it's called.

5. Lambda functions

- Lambda functions are anonymous functions (i.e., functions without a name) that can be passed as parameters or assigned to variables.

6. Higher order function

- Functions that take other functions as parameters or return them as results.

7. Local functions

- Functions defined inside another function. They are only accessible within the outer function.

8. Anonymous functions

- Like lambda expressions but with the full function declaration syntax.

9. Suspend functions

- Functions that can be paused and resumed, used in Kotlin's coroutines for asynchronous programming.
- They are marked with the suspend keyword.

10. Infix functions

- Call the functions in a more natural way without using dot and parenthesis.

- It must have a single parameter without a default value.
- Must be an extension function or a member function.

11. Operator functions

- Used to provide custom implementations for the predefined set of operators.
- Allows you to define or overload operators for custom types.

12. Tail recursive functions

- Optimized by the compiler to prevent stack overflow.

13. Scope functions

- A set of functions that allow you to execute a block of code within the context of an object.

i. let

1. Used to execute a code block containing non null values.
2. Returns the result of its last expression.
3. Object accessible by the reference 'it'.

ii. with

1. Non extension function.
2. Used for calling functions of the object when you don't need to use the returned result.
3. Object accessible by the reference 'this'.

iii. run

1. Like with, useful when you want to call the objects methods.
2. Returns the result of its last expression.
3. Object accessible by the reference 'this'.

iv. apply

1. Used for initializing objects.
2. Returns the object itself.
3. Object accessible by the reference 'this'.

v. also

1. Like apply, useful for performing actions that take the object as an argument.
2. Returns the object itself.
3. Object accessible by the reference 'this'.

Kotlin coroutines

- Kotlin coroutines are powerful feature in Kotlin that allows you to write asynchronous and non-blocking code more easily and efficiently.

Coroutines

- Coroutines are lightweight threads that can be suspended and resumed.
- They are not bound to any particular thread and can switch threads during execution.

Suspend functions

- A suspend function is a function that can be paused and resumed later.
- They are marked with a suspend keyword.

Builders

1. launch

- Starts a new coroutine without blocking the current thread.
- It returns a *Job* which represents the coroutine and can be used to manage its lifecycle.

2. async

- Starts a new coroutine.
- It returns a *Deferred* object which promises a future result.
- It can be used to perform concurrent tasks and retrieve results asynchronously.

3. runBlocking

- Blocks the current thread until its completion.

4. coroutineScope

- Creates a new coroutine scope and suspends the current coroutine until all the launched coroutines in this scope complete.

5. supervisorScope

- Like *coroutineScope*, but the key difference is error handling.
- Child coroutine can fail independently without cancelling the other children.

Dispatchers

- Dispatchers are used to determine the thread on which a coroutine runs.
 1. **Dispatchers.Main**
 - Runs the coroutines on main thread.
 - Suitable for tasks like updating the UI.
 2. **Dispatchers.IO**
 - Optimized for disk or network I/O operations.
 3. **Dispatchers.Default**
 - Optimized for CPU intensive tasks.
 4. **Dispatchers.Unconfined**
 - Start the coroutine in the current thread.
 - But not confined to any specific thread.
 - It will resume in whatever thread is available after suspension.
- If you don't explicitly provide a dispatcher, Kotlin will automatically determine which dispatcher to use based on the context in which coroutine is running.

withContext(Dispatchers.Main)

- Switches the coroutine context and runs the code in the new context.

Processes

- A process is an instance of an application that runs on its own virtual machine with its own memory space.

Threads

- A thread is a smaller unit of execution within a process.
- By default, an Android app runs on a single thread, known as the main thread or UI thread.

Delegation

- Allows an object to transfer some of its responsibilities to another object.
- The 'by' keyword is used to achieve delegation.
 1. Delegate property
 - A property that delegates its getter and setter logic to another object.
 2. Class delegation
 - A mechanism that delegates the implementation of an interface to another object.

Observables

- Properties that can be observed for changes.
- Capability of an object to notify others about changes in its data.

LiveData

- LiveData is an observable data holder class.
- It is lifecycle aware.

Flow

- Flow is a type that can emit multiple values sequentially.
- Built on top of coroutines.

Generics

- Generics allows you to write code that can work with different types without sacrificing type safety.
- It's a way to create classes, interfaces, and functions that can operate on a variety of types while still maintaining compile time type checking.



```
1 class Box<T>(val value: T) {  
2     fun get(): T = value  
3 }  
4  
5 fun <T> singletonList(item: T): List<T> { }  
6  
7 fun <T> T.basicToString(): String { }
```

Variance

- Kotlin provides mechanisms for defining how generic types relate to their subtypes using *in*, *out*, and *where* keywords.

Operators

1. Arithmetic operators (+, -, *, /)
2. Relation operators (>, <, >=, <=, ==)
3. Assignment operators (=, +=, -=, *=, /=)
4. Unary operators (++ , --)
5. Logical operators (&&, ||, !)
6. Bitwise operators (AND, OR, ..)